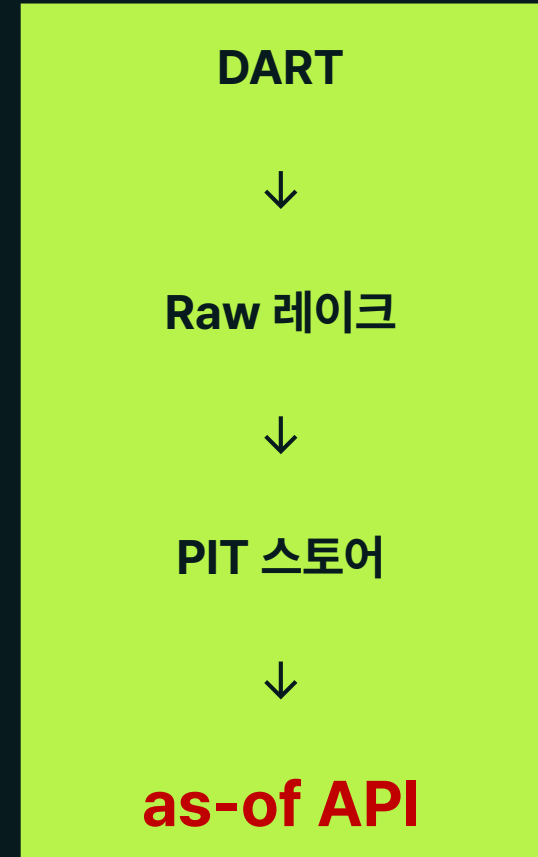


Claude Code로 구축하는 한국 주식 PIT DB

정정공시까지 시점별로 보존하는 데이터 계층

구현 제안 · 데이터베이스 편



네 단계로 구현을 설명합니다

01 ————— 목표와 설계

무엇을 저장하나

데이터 범위 · 아키텍처 · 스키마

02 ————— Claude Code 구성

누가 일하나

CLAUDE.md · 서브에이전트 · hooks

03 ————— 수집과 파싱

어떻게 쌓나

정정 감지 · 원문 파싱 · 로드맵

04 ————— 검증과 운영

어떻게 믿나

회귀 테스트 · 검증 규칙 · cron

에이전트는 코드를 쓰고, 데이터 규칙은 hook이 지킨다

01

목표와 설계

원문 저자가 구축한 범위를 기준으로 삼는다

목표는 1980년 이후 전 종목의 시점별 데이터입니다

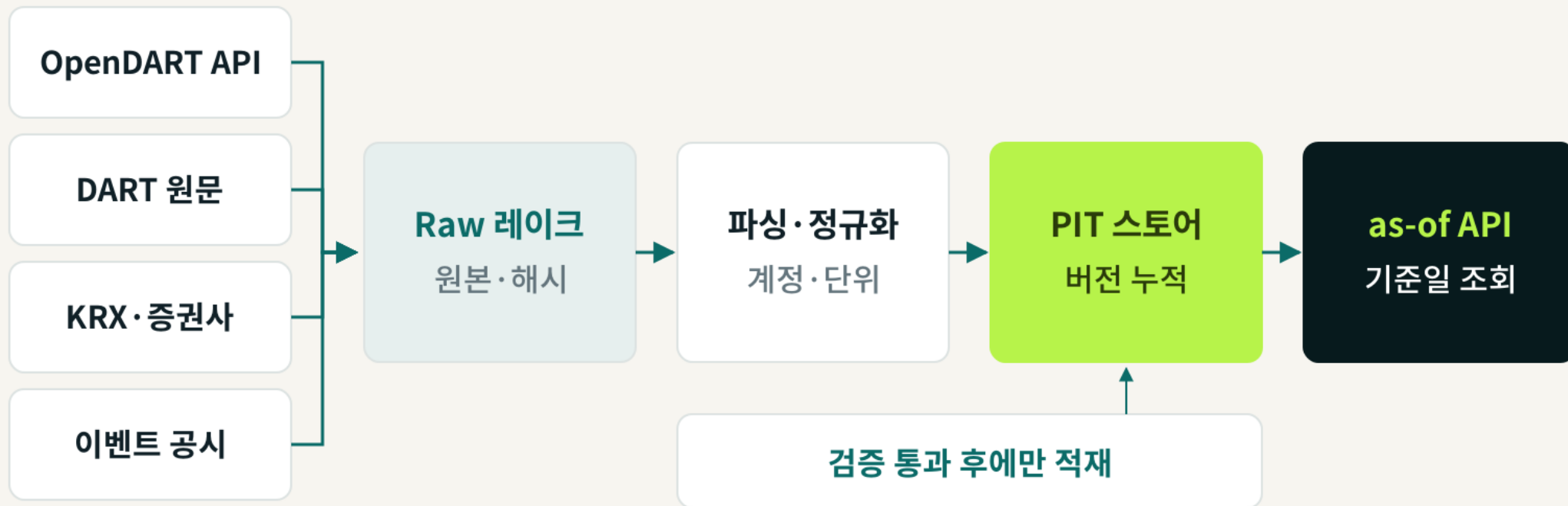
원문 게시물(2026.8)에 공개된 구축 범위

데이터	시작	PIT 키	원천
주가·수정계수·거래대금	1980~	거래일	KRX·증권사
재무제표	1981~	접수일 (1999~)	DART 원문·API
ETF 일별 PDF	2002~	기준일	운용사 공시
배당·증자·합병·상폐	전 기간	공시일	DART 이벤트

모든 행은 '언제 알려졌는가'를 함께 저장합니다

원천에서 as-of API까지 다섯 계층으로 흐릅니다

Raw는 불변, PIT 스토어는 추가만 허용한다



※ 검증 계층을 통과하지 못한 배치는 PIT 스토어에 적재하지 않는다

핵심 테이블은 세 개이고 시점 컬럼이 키입니다

강조 컬럼이 PIT 조회와 생존편향 처리에 쓰인다

fin_pit

corp_code · fs_year

fs_div · account_id

value_krw

rcept_dt · rcept_no

is_amend · source

price_daily

stock_code · trade_dt

close · value_krw

adj_factor

market · sector

halt_flag

corp_event

corp_code · type

announce_dt

merger · split

delist_return

rcept_no

※ 금액은 원 단위 정수(value_krw)로 통일한다

as-of 조회는 기준일 직전 버전 하나만 남깁니다

기준일 이전 접수분 중 최신 버전 = 그 시점에 알 수 있던 값

core/pit.sql — DuckDB

```
-- 기준일 :d 이전 접수분 중 최신 버전만 남긴다
SELECT * FROM fin_pit
WHERE rcept_dt < :d           -- 당일 접수 제외
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY corp_code, fs_year, fs_div, account_id
    ORDER BY rcept_dt DESC, rcept_no DESC) = 1;
```

※ 접수일 당일을 제외해 장 마감 후 공시의 사후참조를 막는다

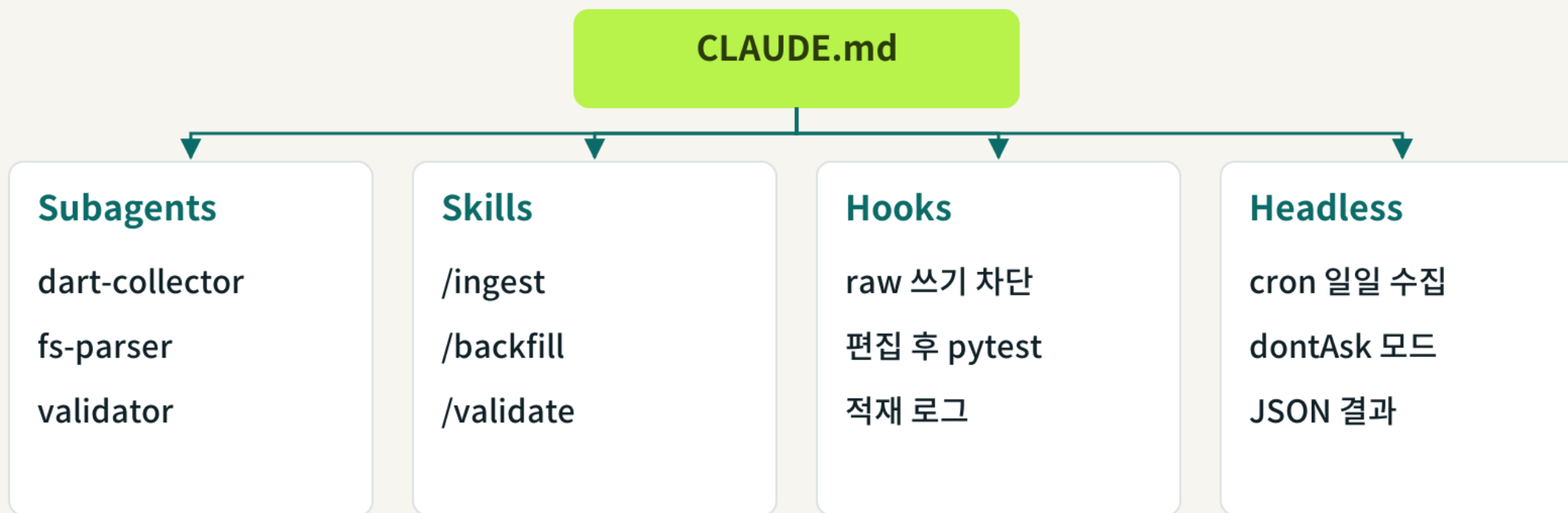
02

Claude Code 구성

프로젝트 기억 · 서브에이전트 · 스킬 · hooks · headless

Claude Code 구성 요소마다 역할을 나눕니다

규칙은 CLAUDE.md, 작업은 서브에이전트, 강제는 hooks가 맡는다



※ 경로는 Claude Code 공식 문서 기준 (code.claude.com/docs)

CLAUDE.md에 불변 규칙과 기준 사례를 적습니다

세션이 시작될 때마다 Claude Code가 읽는 프로젝트 기억

CLAUDE.md

불변 규칙

- data/raw/ 는 읽기 전용 (hook 이 차단)
- fin_pit 은 INSERT 만, 정정은 새 행
- 모든 행에 rcept_dt · rcept_no · source
- 금액은 원 단위 정수 value_krw

작업 순서

- 수집 dart-collector, 파싱 fs-parser
- 적재 전 /validate 통과 필수
- 기준: 대우조선 FY2014 = 330억 (2015-12-31)

※ 형광 줄이 데이터 무결성을 지키는 핵심 규칙

파서는 저가 모델의 서브에이전트로 분리합니다

frontmatter의 model·tools로 비용과 권한을 함께 제한한다

```
.claude/agents/fs-parser.md
```

```
---
```

```
name: fs-parser
```

```
description: 1999~2014 원문 재무표를 계정코드로 매핑
```

```
tools: Read, Write, Bash(python *), Bash(pytest *)
```

```
model: haiku
```

```
---
```

- 단위(천원·백만원)를 읽어 value_krw 로 환산
- 불확실한 계정은 unmapped.csv 에 남긴다
- 연결·개별은 fs_div 로 구분한다

※ 반복 파싱은 haiku, 설계·리뷰는 상위 모델이 맡는다

hook은 규칙을 권고가 아니라 강제로 만듭니다

PreToolUse가 exit 2를 반환하면 해당 도구 호출이 차단된다

.claude/settings.json + guard_raw.sh

```
"PreToolUse": [{ "matcher": "Edit|Write|Bash",  
  "hooks": [{ "type": "command",  
    "command": ".claude/hooks/guard_raw.sh" }] }]
```

```
# guard_raw.sh
```

```
p=$(jq -r ".tool_input.file_path // .tool_input.command")  
[[ "$p" == *data/raw/* ]] && { echo "raw 불변" >&2; exit 2; }  
exit 0
```

※ 편집 후에는 PostToolUse가 테스트를 자동 실행한다

03

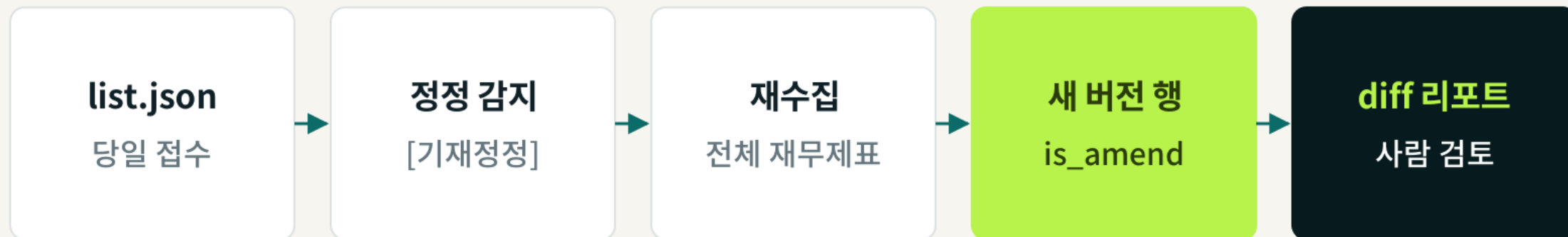
수집과 파싱

정정공시를 새 버전으로 쌓는 흐름

정정공시는 매일 감지해 새 버전 행으로 쌓습니다

OpenDART 공시검색(list.json)과 전체 재무제표 API를 조합한다

매일 18:00 · claude -p



기존 행은 그대로 두고 새 버전만 추가한다

※ 전체 재무제표 API는 2015 사업연도 이후만 제공한다

1999~2014년은 공시 원문을 파싱해야 합니다

API가 없는 구간 — 원문 문서(document.xml)를 내려받아 표를 해석한다

STEP 1

원문 수집

- 보고서·정정 전부
- Raw에 원본 보존

STEP 2

표 파싱

- 단위 표기 판독
- 개별·연결 구분

STEP 3

계정 매핑

- 표준 계정코드로
- 불확실은 보류

※ K-GAAP 시기 계정명은 회사마다 달라 매핑 사전과 사람 검토가 필요하다

추측으로 채운 계정은 PIT의 신뢰를 무너뜨립니다

구축은 다섯 단계로 나누고 게이트로 넘깁니다

단계마다 통과 조건을 정해 두고 자동 검증한다



◆ 게이트 — 통과해야 다음 단계로 넘어간다 (기준은 제안)

04

검증과 운영

실제 정정 사례를 테스트로 고정한다

대우조선 사례를 as-of 회귀 테스트로 고정합니다

원문 스크린샷의 접수일·금액이 기대값이다

tests/test_asof_dsme.py

```
CASES = [    # 대우조선 FY2014 순이익 (억 원)
    ("2015-12-31", 330),    # v1 2015-03-31
    ("2016-12-31", -8631),  # v2 2016-04-14
    ("2017-12-31", -8527),  # v3 2017-05-11
]

@pytest.mark.parametrize("d, exp", CASES)
def test_dsme(d, exp):
    v = asof("042660", 2014, "net_income", d, "CFS")
    assert round(v / 1e8) == exp
```

※ 파서·스키마를 바꿀 때마다 이 테스트가 먼저 통과해야 한다

적재 전 다섯 가지 검증을 통과해야 합니다

validator 서버에이전트가 /validate 스킴으로 실행한다

검증	규칙	실패 시
회계 항등식	자산 = 부채 + 자본	배치 중단
단위 이상치	전기 대비 1,000배 변화	사람 검토
정정 diff	주요 계정 부호 반전	리포트 발송
벤더 대조	DataGuide와 차이 원인	사유 기록
생존편향	상폐 종목 수익률 존재	배치 중단

검증 실패는 숨기지 않고 로그와 리포트로 남깁니다

일일 수집과 검증은 cron에서 headless로 돌립니다

claude -p · dontAsk 모드 · JSON 출력으로 무인 실행

crontab — claude -p

평일 18:00 수집 · 19:00 검증

```
0 18 * * 1-5 claude -p "/ingest today" \
```

```
--permission-mode dontAsk \
```

```
--output-format json > logs/ingest.json
```

```
0 19 * * 1-5 claude -p "/validate today" \
```

```
--permission-mode dontAsk --output-format json
```

```
# JSON 의 total_cost_usd 로 비용을 기록
```

※ dontAsk 모드는 허용 목록 밖의 도구 호출을 모두 거부한다

RAW IS IMMUTABLE.

VERSIONS ARE APPENDED.

AS-OF IS TESTED.

규칙은 hook으로 강제하고
정정 사례는 테스트로 고정합니다.